

LAPORAN PRAKTIKUM
APLIKASI BERBASIS PLATFORM

MODUL 89 PERTEMUAN 12
API PERANGKAT KERAS



Disusun oleh:

Luthfi Adi Harianto

2311102172

S1 IF-11-REG04

Dosen Pengampu:

Cahyo Prihantoro, S.Kom., M.Eng

PROGRAM STUDI S1 INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026

DASAR TEORI

[MODUL PRAKTIKUM Pemrograman Perangkat Bergerak 2024.pdf](#)

LINK GITHUB

[**https://github.com/LuthfiAdiHariantto/Aplikasi-Berbasis-Platform.git**](https://github.com/LuthfiAdiHariantto/Aplikasi-Berbasis-Platform.git)

LATIHAN KELAS – GUIDED 1. GUIDED 1

Source Code

```
import 'dart:io';
import 'package:flutter/foundation.dart'; // Tambahkan ini untuk kIsWeb
import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';
import
'package:flutter_local_notifications/flutter_local_notifications.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Tugas Kamera & Notifikasi',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(primarySwatch: Colors.indigo, useMaterial3: true),
      home: const HomePage(),
    );
  }
}

class HomePage extends StatefulWidget {
  const HomePage({super.key});

  @override
  State<HomePage> createState() => _HomePageState();
}

class _HomePageState extends State<HomePage> {
  // Ubah tipe data dari File? menjadi XFile? agar aman di Web
  XFile? _imageFile;

  final ImagePicker _picker = ImagePicker();
  // Gunakan kata kunci 'dynamic' untuk membypass pengecekan error di Web
  final dynamic _notificationsPlugin =
    FlutterLocalNotificationsPlugin();
  @override
  void initState() {
    super.initState();
    _initNotification();
  }

  // -----
  // Inisialisasi plugin notifikasi lokal (Aman untuk web)
  // -----

  Future<void> _initNotification() async {
    // Lewati inisialisasi jika berjalan di Web
    if (kIsWeb) return;

    const AndroidInitializationSettings androidSettings =
      AndroidInitializationSettings('@mipmap/ic_launcher');
```

```

const InitializationSettings initSettings = InitializationSettings(
  android: androidSettings,
);

await _notificationsPlugin.initialize(initSettings);
}

// -----
// Menampilkan notifikasi lokal
// -----

Future<void> _showNotification() async {
  // Jika di web, tampilkan SnackBar sebagai ganti notifikasi sistem
  if (kIsWeb) {
    if (mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Foto Berhasil ditambahkan!')),
      );
    }
    return;
  }

  const AndroidNotificationDetails androidDetails =
    AndroidNotificationDetails(
      'foto_channel_id',
      'Notifikasi Foto',
      channelDescription: 'Notifikasi setelah foto berhasil
diambil/dipilih',
      importance: Importance.high,
      priority: Priority.high,
    );

  const NotificationDetails notifDetails = NotificationDetails(
    android: androidDetails,
  );

  await _notificationsPlugin.show(
    0,
    'Foto Berhasil!',
    'Foto kamu sudah berhasil diambil dan ditampilkan di aplikasi.',
    notifDetails,
  );
}

// -----
// Fungsi untuk membuka kamera
// -----

Future<void> _ambilFotoDariKamera() async {
  final XFile? hasilFoto = await _picker.pickImage(
    source: ImageSource.camera,
    imageQuality: 80,
  );

  if (hasilFoto != null) {
    setState(() {
      _imageFile = hasilFoto; // Simpan sebagai XFile
    });
    await _showNotification();
  }
}

```

```

// -----
// Fungsi untuk memilih foto dari galeri
// -----

Future<void> _pilihFotoDariGaleri() async {
  final XFile? hasilFoto = await _picker.pickImage(
    source: ImageSource.gallery,
    imageQuality: 80,
  );

  if (hasilFoto != null) {
    setState(() {
      _imageFile = hasilFoto; // Simpan sebagai XFile
    });
    await _showNotification();
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Kamera & Notifikasi'),
      centerTitle: true,
    ),
    body: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Column(
        children: [
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceEvenly,
            children: [
              ElevatedButton.icon(
                onPressed: _ambilFotoDariKamera,
                icon: const Icon(Icons.camera_alt),
                label: const Text('Ambil Foto'),
              ),
              ElevatedButton.icon(
                onPressed: _pilihFotoDariGaleri,
                icon: const Icon(Icons.photo_library),
                label: const Text('Pilih Galeri'),
              ),
            ],
          ),
          const SizedBox(height: 24),
          Expanded(
            child: Container(
              width: double.infinity,
              decoration: BoxDecoration(
                border: Border.all(color: Colors.grey),
                borderRadius: BorderRadius.circular(12),
              ),
              clipBehavior: Clip.hardEdge,
              child: _imageFile == null
                ? const Center(child: Text('Belum ada foto yang
dipilih'))
                : kIsWeb
                  ? Image.network(_imageFile!.path, fit:
BoxFit.contain)

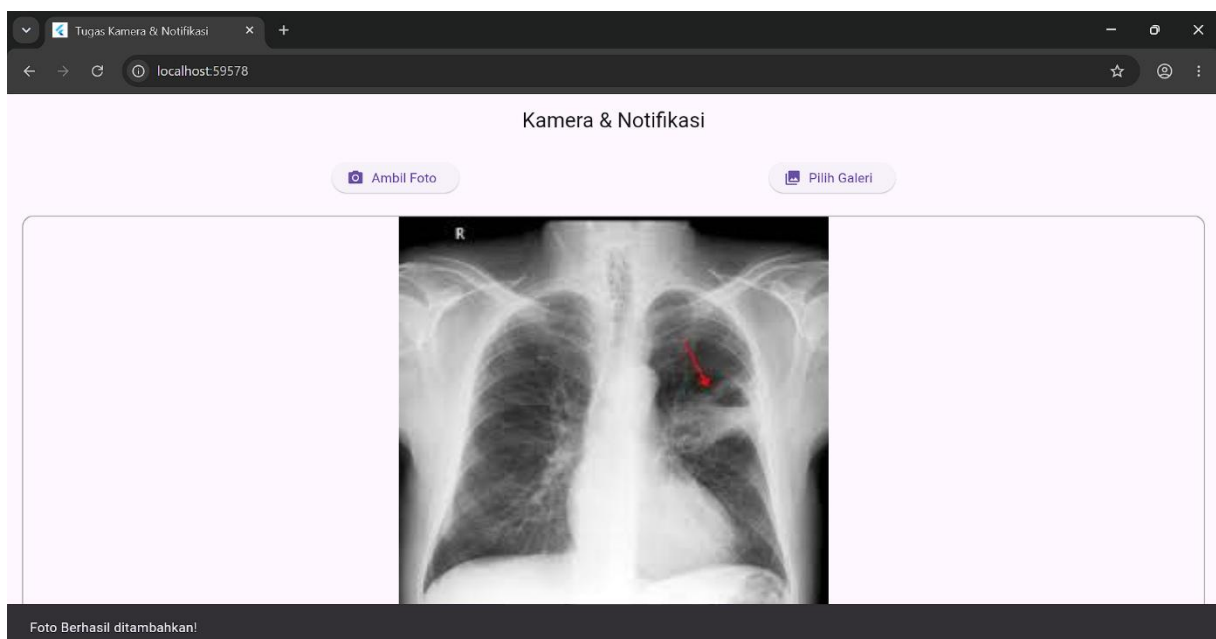
```

```

BoxFit.contain),
        ),
    ],
),
),
);
}
}
: Image.file(File(_imageFile!.path), fit:

```

Hasil



Penjelasan

1. Manajemen Media dan Arsitektur Data

- *Uint8List? _bytesGambar*: Untuk memastikan kompatibilitas pemrosesan data gambar di berbagai lingkungan deployment, berkas media tidak dibaca berdasarkan alamat direktori mentah (file path), melainkan dikonversi dan disimpan ke dalam bentuk biner atau larik data memori (Uint8List). Pendekatan ini menjamin

perenderaan gambar menjadi lebih cepat dan aman dari kegagalan pembacaan hak akses penyimpanan eksternal.

- *CameraController*: Komponen utama yang bertugas sebagai pengontrol aliran data (stream controller) dari perangkat keras penangkap gambar (optical hardware). Komponen ini mengelola proses inisialisasi lensa, penangkapan gambar, hingga pengosongan alokasi memori (disposing) setelah perangkat selesai digunakan.

2. Penjelasan Alur Fungsi Utama

- *Fungsi Kamera (_aktifkanWebcam)*: Fungsi ini mendeteksi subsistem kamera aktif yang tersedia pada perangkat melalui perintah `availableCameras()`. Sistem kemudian mengaktifkan lensa utama dengan tingkat resolusi menengah (`ResolutionPreset.medium`). Ketika kamera berhasil diinisialisasi, variabel penanda `_isKameraAktif` berubah menjadi `true` untuk memicu pemunculan jendela pratinjau visual secara langsung (live preview).
- *Fungsi Tangkap Gambar (_tangkapFotoWebcam)*: Saat pengguna melakukan penangkapan gambar (capture), fungsi `takePicture()` memproses pembekuan frame video. Hasil tangkapan tersebut langsung diekstrak ke dalam representasi biner (`readAsBytes()`) untuk disimpan ke dalam variabel memori, dan kontroler perangkat keras segera dimatikan untuk efisiensi performa aplikasi.
- *Fungsi Akses Galeri (_pilihFotoDariGaleri)*: Fungsi ini memanggil antarmuka sistem operasi untuk membuka jendela penjelajah berkas media (Galeri). Berkas gambar yang dipilih oleh pengguna akan langsung dibaca datanya dalam bentuk array byte agar memiliki format yang seragam dengan hasil tangkapan kamera.

3. Komponen Widget UI yang Digunakan

- *CameraPreview*
Widget khusus yang berfungsi untuk merender atau memproyeksikan siaran video langsung (live feed) dari lensa kamera aktif secara real-time ke dalam tata letak aplikasi.
- *Image.memory()*
Widget yang digunakan khusus untuk mendekode dan menampilkan visualisasi gambar berdasarkan sumber data biner (`Uint8List`) yang tersimpan di dalam memori lokal aplikasi.
- *SnackBar (Sistem Notifikasi)*
Bertindak sebagai media komunikasi utama untuk mengirimkan umpan balik (feedback) kepada pengguna. Ditampilkan dengan mode melayang (`SnackBarBehavior.floating`) di area bawah layar untuk menginformasikan status keberhasilan sistem secara instan, baik saat kamera berhasil menangkap gambar maupun saat berkas galeri berhasil dimuat.
- *ClipRRect*
Widget layouting yang berfungsi untuk memotong atau menghaluskan sudut-sudut tajam pada komponen visual (termasuk tampilan kamera dan gambar) agar berbentuk melengkung (rounded corners), memberikan kesan antarmuka yang rapi dan modern.
- *ElevatedButton.icon*
Komponen tombol interaktif dengan efek timbul (elevasi). Tombol ini dikondisikan secara dinamis; akan berwarna hijau toska bertuliskan "Buka Kamera" untuk memulai

inisialisasi, dan otomatis berubah fungsi menjadi tombol merah bertuliskan "Ambil Foto Sekarang (Capture)" ketika aliran kamera sedang aktif berjalan.

Oiya jangan lupa tambahkan dependensi *image_picker* dan *camera* di *pubspec.yaml*

***flutter pub add image_picker flutter
pub add camera***

sekaligus tambahkan import packagenya juga ya di awal file *main.dart*

tidak bisa menggunakan android studio karena keterbatasan pada perangkat